

From Chains to Cycles: A Practical Guide to Building Standard RAG and GraphRAG Agents with LangGraph

Architectural Blueprint and Code Implementation for Standard RAG

The development of a Retrieval-Augmented Generation (RAG) system within the specified technology stack—a Python backend using FastAPI, PostgreSQL with pgvector for storage, and LangChain for orchestration—requires a multi-faceted approach that integrates data persistence, natural language processing, and web service frameworks. A production-ready standard RAG system is not merely a linear pipeline but an engineered solution where each stage, from document ingestion to answer generation, must be robust and efficient [12](#). The architectural blueprint for such a system can be deconstructed into four primary phases: Ingestion and Chunking, Embedding and Indexing, Retrieval, and finally, Augmentation and Generation. Each phase leverages specific components from the user's stack to build a cohesive and powerful question-answering application. The integration of FastAPI provides the necessary interface for external clients to interact with this system, while SQLAlchemy and pgvector serve as the durable backbone for storing and retrieving information at scale [9](#) [18](#).

The initial phase, Ingestion and Chunking, addresses how raw textual data is prepared for embedding. It is a critical step where naive approaches, such as splitting documents uniformly at 512 tokens, are often insufficient for engineering-grade applications [10](#). A more sophisticated strategy involves using advanced text splitters like the `RecursiveCharacterTextSplitter`, which intelligently breaks down text based on a hierarchy of separators (e.g., newlines, sentences, spaces), preserving logical document boundaries. This ensures that when chunks are retrieved during a query, they contain coherent pieces of information, which significantly improves the quality of the final generated response. Following chunking, the process moves to Embedding and Indexing. Here, each document chunk is converted into a high-dimensional vector representation using an embedding model, such as OpenAI's `text-embedding-3-large`. These vectors capture the semantic meaning of the text. The LangChain library provides a direct template for integrating with `pgvector`, allowing these vectors to be stored efficiently

within a PostgreSQL database [18](#). Using `SQLModel`, one can define a clear data model for the document chunks, including columns for the raw text, metadata (like source file name or page number), and a specialized `Vector` column type provided by `pgvector` to store the embeddings [22](#). This creates a structured yet flexible schema for managing a corpus of knowledge.

The third phase, Retrieval, is triggered when a user submits a query. The same embedding model used during indexing is applied to the user's input, converting the query into a query vector. A similarity search is then performed against the `pgvector` index in the database to find the most semantically relevant document chunks [12](#). This process retrieves a set of contextually rich snippets that are likely to contain the information needed to answer the user's question. Finally, the Augmentation and Generation phase synthesizes the retrieved information. The top-k retrieved chunks are formatted into a prompt alongside the original user query. This augmented prompt is designed to guide a Large Language Model (LLM), such as GPT-4o accessed via the `langchain-openai` package, to generate a concise, accurate, and factually grounded answer based on the provided context [5](#). The entire workflow is orchestrated using LangChain's modern paradigm, LCEL (LangChain Expression Language), which allows for the declarative and modular composition of these stages into a single, reusable chain [14](#). This architecture forms the foundation for a powerful and scalable RAG application.

The following table summarizes the key architectural components and their corresponding implementation details within the specified technology stack for a standard RAG system.

Component	Technology/ Concept	Description & Implementation Details
Backend Framework	FastAPI	Provides the RESTful API layer for the application. Endpoints will receive user queries, invoke the RAG chain, and stream responses back to the client. Pydantic models (pydantic>=2.13.4) are used for request/response data validation 7 9 .
Data Persistence	PostgreSQL with pgvector	The primary database for storing document chunks and their vector embeddings. The SQL-pgvector template from LangChain is used to interact with the database, leveraging the pgvector extension for efficient similarity search 18 22 .
ORM Layer	SQLModel	Used to define the data models for the database tables. A DocumentChunk model would include fields for the text content, metadata, and a Vector field to interface with the pgvector column 4 .
Retrieval Logic	Similarity Search	When a query is received, its embedding is computed and used to perform a KNN (k-nearest neighbors) search on the pgvector index to retrieve the most relevant document chunks 12 .
Orchestration	LangChain (≥ 1.3.4)	Manages the end-to-end pipeline. The langchain-community package may provide loaders for ingesting various document types, and the langchain-openai package provides the ChatOpenAI model for generation 3 14 .
Embeddings	OpenAI Embeddings	Utilizes models like text-embedding-3-large from the langchain-openai package to convert both document chunks and user queries into numerical vectors for semantic comparison 1 .

To implement this architecture, one begins by defining the database schema using SQLModel. An example DocumentChunk class would look like this:

```
from sqlmodel import Field, SQLModel
from typing import Optional

class DocumentChunk(SQLModel, table=True):
    __tablename__ = "document_chunks"
    id: Optional[int] = Field(default=None, primary_key=True)
    text: str = Field(sa_type=sa.Text)
    metadata_: str = Field(alias="metadata", sa_column=sa.Column("metadata"))
    vector: Optional[List[float]] = Field(default=None, sa_type=Vector(1536))
```

Next, an ingestion script processes source documents. It uses a loader from langchain_community, splits the text using a RecursiveCharacterTextSplitter, generates embeddings with OpenAIEmbeddings, and inserts the chunks into the PostgreSQL database via SQLAlchemy sessions managed by SQLModel.

The core of the RAG logic is the retrieval-augmented generation chain built with LCEL. This involves creating a retriever object, typically from the pgvector-enabled database, and connecting it to a prompt template and an LLM.

```

from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain

llm = ChatOpenAI(model="gpt-4o")
prompt_template = """Use the given context to answer the question.
If you don't know the answer, just say that you don't know.
Context: {context}
Question: {input}"""
prompt = PromptTemplate.from_template(prompt_template)

# Assuming 'retriever' is a configured retriever object pointing to pgvector
document_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, document_chain)

```

Finally, this chain is integrated into the FastAPI application. A route is created to accept a POST request containing the user's question. Inside the route handler, the `rag_chain` is invoked asynchronously, and the streaming output from the LLM is handled using `sse-starlette` to provide a real-time response to the user [7](#). This complete integration demonstrates a robust, end-to-end standard RAG implementation tailored to the user's specified technology stack, providing a solid foundation for a private knowledge base or question-answering system.

Architectural Blueprint and Code Implementation for GraphRAG

The evolution from standard RAG to GraphRAG represents a significant architectural advancement aimed at tackling the limitations of linear, token-based retrieval. While standard RAG excels at finding surface-level evidence within documents, it struggles with complex, relational queries that require understanding connections across multiple documents, such as summarizing all risks mentioned in different project reports [15](#). GraphRAG, pioneered by Microsoft, addresses this by first constructing a knowledge graph from the document corpus and then leveraging this graph structure to inform the retrieval process [15](#). This approach fundamentally shifts the problem from a simple similarity search to a more nuanced exploration of entity and relationship networks. For implementation, the user's stack, particularly LangGraph $\geq 1.2.4$, becomes indispensable.

LangGraph is designed to model complex, stateful workflows as directed cyclic graphs (DCGs), making it the ideal framework for orchestrating the multi-step, iterative, and conditional logic inherent in a GraphRAG system [11](#) [16](#) [17](#).

The architecture of a GraphRAG system is more intricate than its standard counterpart and can be broken down into several key stages. The first stage is Knowledge Extraction, where an LLM is prompted to analyze the unstructured text of the documents and extract meaningful entities (e.g., people, organizations, concepts) and the relationships between them. This results in a set of triples, typically in the form of (subject, predicate, object). For example, from the sentence "Project Phoenix was initiated by Alice," the system might extract the triples (Project Phoenix, has_initiator, Alice) and (Alice, leads, Project Phoenix). This step transforms unstructured text into structured, machine-readable knowledge.

The second stage is Knowledge Graph Construction. The extracted triples are then used to populate a knowledge graph. While dedicated graph databases like Neo4j are common, the user's stack suggests a relational representation within PostgreSQL. This could involve creating tables for nodes (entities) and edges (relationships), linking them via foreign keys. Alternatively, a more flexible approach could use JSONB fields within a single table to store the graph structure. This graph serves as a high-level map of the domain knowledge contained within the documents. The third stage is Query Decomposition and Expansion. When a user poses a complex query, the system does not immediately perform a vector search. Instead, it first analyzes the query to identify key entities and intent. It then uses these entities to traverse the constructed knowledge graph, identifying related nodes and paths. This traversal helps to expand the initial query, uncovering relevant information that might be spread across many different documents. For instance, a query about "risks" might lead the system to traverse the graph from a project node to its associated risk nodes, and then to the person responsible for those risks.

The fourth stage is Hybrid Retrieval. To ensure comprehensive context gathering, the system performs a hybrid search. It combines the results from the graph traversal—which provides structurally and relationally relevant information—with a traditional vector similarity search on the document chunks. This dual approach ensures that the final context fed to the LLM is both semantically similar to the query and structurally connected through the knowledge graph. The final stage is Generation, where the rich, multi-faceted context gathered from the hybrid retrieval is formatted into a prompt for an LLM, which then synthesizes a coherent and insightful answer. The entire workflow, involving extraction, construction, querying, and generation, is perfectly suited to be modeled as a LangGraph application. Each major step becomes a node in the graph, and

the explicit state control allows for managing the flow of information (e.g., the user query, extracted entities, graph structure, retrieved documents) between these nodes [6](#).

Implementing this architecture requires defining a custom LangGraph state and building out the corresponding nodes. The state can be defined using a TypedDict to hold all the intermediate data.

```
from typing import Annotated, List, Dict, Any
from langgraph.graph import StateSchema
```

```
class GraphState(StateSchema):
    user_query: str
    entities: List[str]
    relations: List[Dict[str, str]]
    graph_structure: Dict[str, List[str]] # e.g., {"Project_A": ["Risk_X",
    retrieved_documents: List[str]
    answer: str
```

Each function in the workflow becomes a "node" in the graph. For example:

- ``extract_knowledge_node(state):`` Takes the user query, calls an LLM to extract entities and relations, and updates the state.
- ``build_graph_node(state):`` Takes the extracted knowledge and constructs the graph representation (e.g., a dictionary or a networkx graph), updating the state.
- ``query_graph_node(state):`` Uses the graph structure to perform a traversal based on the user query, retrieving relevant entities and expanding the search scope.
- ``retrieve_documents_node(state):`` Takes the expanded search terms from the graph traversal and performs a vector search on the pgvector index to fetch the actual document chunks.
- ``generate_answer_node(state):`` Combines the retrieved documents and the original query, prompts the LLM to generate the final answer, and stores it in the state.

Edges between nodes determine the flow. Initially, the graph would proceed sequentially from `extract_knowledge_node` to `build_graph_node` and so on. However, LangGraph's power lies in its ability to introduce loops and conditions. For example, if the graph traversal yields no results, an edge could loop back to refine the query or fall back to a simpler vector-only retrieval path. This explicit, visual, and programmable control over the agent's thought process is what distinguishes LangGraph from simpler chaining mechanisms and makes it the cornerstone of a robust GraphRAG implementation. While a full, production-ready tutorial for this specific pattern may not

exist in the provided sources, this blueprint provides a clear, actionable plan for building such a system, directly leveraging the advanced capabilities of LangGraph $\geq 1.2.4$ to manage the complexity of knowledge-intensive tasks.

Reference

1. Agent/RAG黑科技的人话说明书+通关路线图 - 知乎专栏 <https://zhuanlan.zhihu.com/p/1917949732053030804>
2. Mast3r-SLAM updated to Rerun with Mojo backend - LinkedIn https://www.linkedin.com/posts/pablo-vela_ive-migrated-the-old-mast3r-slam-example-activity-7448233418464845824-lRk8
3. [PDF] Generative AI for Official Statistics - UNECE <https://unece.org/sites/default/files/2025-09/Generative%20AI%20for%20Official%20Statistics%20HLG-MOS%20Report.pdf>
4. Analyze pip install ERROR: Could not find a version that satisfies the ... <https://discuss.python.org/t/analyze-pip-install-error-could-not-find-a-version-that-satisfies-the-requirement/55006>
5. [PDF] Unleashing the Power of AI with IBM watsonx.ai <https://www.redbooks.ibm.com/redbooks/pdfs/sg248574.pdf>
6. Wassim EL BAKKOURI's Post - LinkedIn https://www.linkedin.com/posts/wassim-elbakkouri_fastapi-langgraph-langchain-activity-7393546592756211714-5qbS
7. 【流程实操】RAG+LangChain+FastAPI+OpenAI+通义千问打造私有 ... <https://www.bilibili.com/video/BV1GmWrexE5H/>
8. Microsoft Ignite 2025 Book of News <https://news.microsoft.com/ignite-2025-book-of-news/>
9. AI-Agent入门实战-AI私厨原创 - CSDN博客 https://blog.csdn.net/2301_80335585/article/details/160960992
10. Knowledge Graphs for LLM-Based Industrial Asset Operations https://www.linkedin.com/posts/samyamaa_knowledgegraphs-llmagents-industrialai-activity-7465360795137200128-fVCn
11. 一文带你看懂AI Agent 智能体 - DAMO开发者矩阵 <https://damodev.csdn.net/69436777836da3214485a5a9.html>

12. Vector Storage Design for Efficient Retrieval | Himanshu Verma ... https://www.linkedin.com/posts/himanshu-verma-822a07286_ai-llm-vectordb-activity-7457432522122252288-TqLi
13. 一文带你弄懂AI 圈爆火的新概念: Harness Engineering 原创 https://blog.csdn.net/m0_59164520/article/details/160418284
14. 人工智能之智能体应用第一章大模型应用开发基础框架入门 - CSDN博客 https://blog.csdn.net/qq_39434761/article/details/161089323
15. AI技术如何重塑你的工作与行业 - AtomGit开源社区 <https://gitcode.csdn.net/69c9fadb54b52172bc65755f.html>
16. What is LangGraph? - IBM <https://www.ibm.com/think/topics/langgraph>
17. LangChain vs LangGraph: A Developer's Guide to Choosing Your AI ... <https://milvus.io/blog/langchain-vs-langgraph.md>
18. SQL-pgvector | 🦜 LangChain - AiDocZh <https://www.aidoczh.com/langchain/v0.2/docs/templates/sql-pgvector/>
19. PicoClaw: AI on \$10 Hardware with 1 Second Boot - LinkedIn https://www.linkedin.com/posts/saifalramahi_10-hardware-10mb-ram-1-second-boot-an-activity-7428791015488053248-6Dm0
20. AI tool updates GitHub README files | Idris Fagbemi posted on the ... https://www.linkedin.com/posts/idris-fagbemi-417a559b_ai-devsecops-langchain-activity-7338819839953645569-5XR8
21. Super excited! | Shirin Khosravi Jam - LinkedIn https://www.linkedin.com/posts/shirin-khosravi-jam_super-excited-week-1-of-moai-rag-systems-activity-7359134491669422081-JLd5
22. AI in .NET - RAG System with Postgres and Ollama | Stefan Đokić https://www.linkedin.com/posts/djokic-stefan_ai-in-net-rag-system-with-postgres-and-activity-7404076991441899520-k9k4